

南開大學
Nankai University

计算机学院

自然语言处理实验报告

文本表示、词向量与 BERT 文本分类

姓名： 牛帅

学号： 2312659

专业： 计算机科学与技术

课程： 自然语言处理

10th October 2026

目录

1 实验要求与实验目标 **3**

2 数据集与统一实验设置 **3**

2.1 数据集说明 3

2.2 数据划分与可复现性 4

2.3 评价指标 4

3 实验环境与实现方案 **4**

3.1 软硬件环境 4

3.2 统一实现原则 5

4 Task 1：Bag-of-Words 文本表示 **5**

4.1 方法原理 5

4.2 Binary Bag of Words 6

4.3 Word Frequency 6

4.4 TF-IDF 6

4.5 实验结果与分析 6

4.6 关键实现代码 6

5 Task 2：Word Embedding 文本表示 **7**

5.1 文档向量构造 7

5.2 预训练 GloVe 8

5.3 AG News Word2Vec 8

5.4 NYT Word2Vec 8

5.5 实验结果与分析 8

5.6 关键实现代码 8

6 Task 3：预训练 BERT 微调 **9**

6.1 模型与训练流程 9

6.2 训练过程 10

6.3 测试结果与错误分析 10

6.4 关键实现代码 11

7 综合比较与讨论 **12**

8 心得体会 **12**

9 实验结论 **13**

A 代码与结果文件 **13**

1 实验要求与实验目标

本实验围绕监督式新闻文本分类，要求在同一数据划分和同一评价标准下，完成三类具有代表性的文本表示与分类实验：

1. 使用 Binary Bag-of-Words、Word Frequency 和 TF-IDF 三种稀疏词袋表示，训练分类器并比较结果；
2. 使用预训练 GloVe、在 AG News 语料上训练的 Word2Vec、以及仅在 NYT 训练集上训练的 Word2Vec 构造文档向量，完成分类实验；
3. 使用 `google-bert/bert-base-uncased` 对分类模型进行端到端微调，记录训练过程、验证集性能和测试集结果。

实验的目标是理解不同文本表示的建模假设及其适用范围，掌握从数据划分、特征构造、模型训练到指标计算的完整流程，并能够依据实验数据分析类别不平衡、词汇覆盖率、上下文建模能力和过拟合等因素对分类性能的影响。实验代码、固定划分、逐条预测和汇总结果保存在仓库的 `scripts/`、`results/` 等目录中。

2 数据集与统一实验设置

2.1 数据集说明

实验使用 NYT 新闻数据集的三个主题类别：*business*（商业）、*politics*（政治）和 *sports*（体育）。每条样本由一段新闻文本和一个类别标签组成。数据集共包含 11,519 条样本，类别分布如表 1 所示。

表 1: NYT 数据集类别分布

类别	样本数	占比
business	1,429	12.41%
politics	1,451	12.60%
sports	8,639	74.99%
总计	11,519	100.00%

数据具有明显的类别不平衡：sports 样本约占四分之三，而 business 和 politics 样本数量接近。因而报告同时使用 Accuracy 和 Macro-F1，避免只依据多数类的识别效果判断模型优劣。

2.2 数据划分与可复现性

所有方法使用完全相同的分层随机划分，随机种子固定为 42。训练集、验证集和测试集的规模与比例见表 2。测试集只在模型选择完成后使用一次。

表 2: 固定数据划分

子集	样本数	比例	用途
Training Set	9,215	80%	训练特征、训练分类器
Validation Set	1,152	10%	选择 BERT 最佳轮次
Test Set	1,152	10%	最终性能报告
总计	11,519	100%	

2.3 评价指标

设测试集中共有 N 条样本，指示函数 $\mathbb{I}(\cdot)$ 在预测正确时取 1，则 Accuracy 定义为

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{y}_i = y_i). \tag{1}$$

对于第 c 个类别，Precision、Recall 和 F1 分别为

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{2}$$

三分类实验报告 Macro-F1：

$$\text{Macro-F1} = \frac{1}{3}\sum_{c=1}^{3}F1_c. \tag{3}$$

Macro-F1 对三个类别赋予相同权重，更能反映少数类 business 和 politics 的识别质量。

3 实验环境与实现方案

3.1 软硬件环境

实验环境如表 3 所示。Task 1 和 Task 2 使用 scikit-learn[4] 与 Gensim，Task 3 使用 PyTorch 和 Hugging Face Transformers[6]，并在 GPU 上进行混合精度训练。

表 3: 实验软硬件环境

项目	配置
操作系统	Windows，使用项目虚拟环境 `.venv/`
CPU/GPU	NVIDIA GeForce RTX 4060 Laptop GPU
CUDA 与深度学习框架	CUDA 12.6；PyTorch 2.8.0+cu126
NLP 工具	Transformers 5.3.0；scikit-learn；Gensim
预训练模型	`google-bert/bert-base-uncased`
随机性控制	Python、NumPy、分类器和 Word2Vec 均使用种子 42
精度设置	BERT 使用 fp16 自动混合精度

3.2 统一实现原则

为了使三项任务具有可比性，实验遵循以下原则：

- 所有模型使用相同的训练、验证和测试索引；测试标签只用于最终评估；
- Task 1 的三种稀疏表示统一使用 Logistic Regression 分类器，词表仅从训练文本拟合；
- Task 2 的三种词向量方法均将每条文本转换为 100 维文档向量，再使用 Logistic Regression；
- BERT 的最大输入长度固定为 64 个 token，训练 3 个 epoch，并根据验证集 Macro-F1 选择最佳模型；
- 除汇总指标外，保存每条验证集和测试集预测，利用 `scripts/verify\_results.py` 重新计算指标、混淆矩阵和哈希值。

4 Task 1：Bag-of-Words 文本表示

4.1 方法原理

设训练语料建立的词表为 $V = \{v_1, \ldots, v_{|V|}\}$。词袋模型忽略词语的顺序，只记录文档中词语的出现情况或权重。三种表示的区别在于每个词对应的特征值不同。

4.2 Binary Bag of Words

Binary BoW 只表示词语是否出现：

$$x_j^{\text{bin}}(d) = \begin{cases} 1, & v_j \in d, \\ 0, & v_j \notin d. \end{cases} \tag{4}$$

同一个词在一篇文档中出现一次或多次都会得到相同的特征值。该表示强调词汇存在性，能够减弱长文档或重复词语带来的影响。本实验使用训练集拟合出 66,180 个词特征。

4.3 Word Frequency

Word Frequency 使用词语在文档中的原始出现次数：

$$x_j^{\text{freq}}(d) = \text{count}(v_j, d). \tag{5}$$

与 Binary BoW 相比，该表示保留了词语重复程度，因此可以利用主题词在同一篇新闻中的多次出现。由于不同新闻长度可能不同，频数也可能受到文档长度影响。

4.4 TF-IDF

TF-IDF[2] 同时考虑词语在当前文档中的频率和在整个训练语料中的稀有程度：

$$x_j^{\text{tfidf}}(d) = \text{tf}_{j,d} \cdot \left(\log \frac{1+N}{1+\text{df}_j} + 1 \right), \tag{6}$$

其中 N 为训练文档数，df_j 为包含词 v_j 的训练文档数。常见词的逆文档频率较小，具有区分度的主题词权重相对更高。

4.5 实验结果与分析

三种方法在完整测试集上的结果见表 4。

4.6 关键实现代码

下面的代码摘取展示了三种词袋表示的统一训练流程。实际脚本还包括数据读取、固定划分、模型持久化和逐条预测保存等部分；这里仅保留特征构造、分类器训练和指标计算的核心语句。

Listing 1: Task 1 的词袋特征与 Logistic Regression

```
vectorizers = {
    "binary": CountVectorizer(binary=True),
    "frequency": CountVectorizer(binary=False),
    "tfidf": TfidfVectorizer()
}

for name, vectorizer in vectorizers.items():
    X_train = vectorizer.fit_transform(train_texts)
    X_test = vectorizer.transform(test_texts)
    clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(X_train, train_labels)
    pred = clf.predict(X_test)
    score = f1_score(test_labels, pred, average="macro")
```

表 4: Task 1：三种词袋表示的测试集结果

表示方法	词表维度	Accuracy	Macro-F1
Binary Bag of Words	66,180	0.989583	0.974058
Word Frequency	66,180	0.992188	0.980103
TF-IDF	66,180	0.986111	0.969559

Word Frequency 的 Accuracy 和 Macro-F1 均为三者最高，说明在该数据集上，词语重复次数包含了有用的主题信息。Binary BoW 略低于 Word Frequency，但仍取得很高的性能，表明新闻类别具有较明显的关键词信号。TF-IDF 的结果反而较低，可能原因是 sports 类样本数量较多，许多与体育主题相关的词在训练语料中频繁出现，IDF 对它们进行了较强的削弱；同时，少数类的判别需要利用一些高频但稳定的领域词。由于词袋模型不建模词序和长距离语义，它对否定、实体关系和上下文歧义的处理能力有限。

5 Task 2：Word Embedding 文本表示

5.1 文档向量构造

每个词 w_i 对应一个 100 维词向量 $e(w_i) \in \mathbb{R}^{100}$。对文档中能够找到向量的词取平均，得到文档表示：

$$e(d) = \frac{1}{n} \sum_{i=1}^{n} e(w_i), \tag{7}$$

其中 n 是文档中已知词的数量。若文本中存在未登录词，则忽略该词；如果整篇文本没有已知词，则使用零向量。为了保证公平，三种方法都使用相同的平均池化和 Logistic Regression。

5.2 预训练 GloVe

GloVe 6B 100d 向量在大规模语料上预训练，能够提供较稳定的通用词义和词间关系 [5]。它不使用 NYT 测试标签，也不在本实验测试集上继续训练。测试集词覆盖率为 98.1571%，说明绝大部分词能够转换为语义向量。

5.3 AG News Word2Vec

第二种方法在完整 AG News 语料的 90,000 条训练文本上训练 Word2Vec[3]。使用 Skip-gram，参数为 `vector_size=100`、`window=5`、`min_count=2`、`epochs=5`、`seed=42`、`workers=1`。该方法的训练语料与 NYT 领域不同，但规模较大，能够学习较充分的通用新闻词汇。测试集词覆盖率为 97.0913%。

5.4 NYT Word2Vec

第三种方法只使用 NYT Training Set 的 9,215 条训练文本训练 Word2Vec，仍使用上述参数。它和分类器的训练领域完全一致，同时严格避免使用验证集和测试集文本来学习词向量。测试集覆盖率达到 99.1992%，高于另外两种方法，说明领域内训练有助于覆盖 NYT 的专有词汇。

5.5 实验结果与分析

表 5: Task 2：词向量文档表示的测试集结果

词向量来源	维度	测试集覆盖率	Accuracy / Macro-F1
Stanford GloVe 6B	100	98.1571%	0.985243 / 0.964617
AG News Word2Vec	100	97.0913%	0.974826 / 0.938885
NYT Word2Vec	100	99.1992%	0.982639 / 0.956510

5.6 关键实现代码

词向量实验先对每篇文档进行平均池化，再将得到的固定长度向量交给同一个分类器。以下函数省略了分词和词表加载，仅展示未知词过滤、平均操作以及分类器训练的关

键逻辑。

Listing 2: Task 2 的文档平均词向量

```
def document_vector(tokens, keyed_vectors, dim=100):
    vectors = [keyed_vectors[w] for w in tokens
               if w in keyed_vectors]
    if not vectors:
        return np.zeros(dim, dtype=np.float32)
    return np.mean(vectors, axis=0)

X_train = np.vstack([
    document_vector(tokens, vectors) for tokens in train_tokens
])
X_test = np.vstack([
    document_vector(tokens, vectors) for tokens in test_tokens
])
clf = LogisticRegression(max_iter=1000, random_state=42)
clf.fit(X_train, train_labels)
```

GloVe 的 Macro-F1 最高，说明大规模预训练语料提供了更有区分度的通用语义空间。NYT Word2Vec 的覆盖率最高，但平均词向量仍然略低于 GloVe，表明覆盖率只是必要条件，词向量质量和跨文本的语义组织方式同样重要。AG News Word2Vec 的结果最低，可能与 AG News 和 NYT 的领域、标题风格及类别词分布不同有关。三种方法均采用简单平均，词序和局部组合信息会被压缩，因此性能略低于能够直接建模上下文的 BERT。

6 Task 3：预训练 BERT 微调

6.1 模型与训练流程

本实验使用 `google-bert/bert-base-uncased`，在分类头上接三分类线性层。BERT 的双向预训练思想和微调范式见 Devlin 等人的工作 [1]。输入文本先由 BERT tokenizer 编码，最大长度设为 64，超过部分截断，不足部分补齐。模型通过交叉熵优化：

$$\mathcal{L} = -\log \frac{\exp(z_y)}{\sum_{c=1}^{3} \exp(z_c)}, \tag{8}$$

其中 z_c 是分类头对第 c 类的 logit，y 是真实标签。训练参数为 batch size 16、learning rate 2×10^{-5}、weight decay 0.01、warmup ratio 0.1、最大梯度范数 1.0，共训练 3 个 epoch。每个 epoch 完整遍历 9,215 条训练样本，即 576 个 batch。每轮结束后在验证集计算 Accuracy 和 Macro-F1，保存验证集 Macro-F1 最优的模型。

6.2 训练过程

表 6: BERT 微调过程

Epoch	Train Loss	Validation Accuracy	Validation Macro-F1
1	0.236977	0.960069	0.908322
2	0.052642	0.973958	0.941443
3	0.017626	0.978299	0.951956

训练损失从 0.236977 降至 0.017626，验证集 Accuracy 和 Macro-F1 持续提高，当前 3 个 epoch 内没有出现明显的验证集退化，因此选择第 3 轮作为最终模型。验证集 Macro-F1 的提升幅度小于训练损失的下降幅度，说明模型逐渐拟合训练集，同时泛化收益开始变小，继续增加 epoch 仍需要依靠验证集进行监控。

6.3 测试结果与错误分析

最佳模型在测试集上的 Accuracy 为 0.982639，Macro-F1 为 0.960523。逐类指标如表 7 所示。

表 7: BERT 测试集逐类指标

类别	Precision	Recall	F1	Support
business	0.937931	0.951049	0.944444	143
politics	0.944444	0.937931	0.941176	145
sports	0.996524	0.995370	0.995947	864
Macro average	0.959633	0.961450	0.960523	1,152

混淆矩阵的行是真实标签、列是预测标签，类别顺序为 business、politics、sports：

表 8: BERT 测试集混淆矩阵

真实类别	预测类别		
	business	politics	sports
business	136	6	1
politics	7	136	2
sports	2	2	860

sports 的 F1 达到 0.995947，说明该类别有鲜明的主题词和足够多的训练样本。business 与 politics 之间的混淆更明显：business 有 6 条被预测为 politics，politics 有 7 条被预测为 business，这些文本可能同时包含政府、公司、市场或政策词汇。少量少数类文本被预测为 sports，通常与球队、赛事或经济新闻中的体育实体有关。由于 Macro-F1 对少数类同样计权，business 和 politics 的少量错误仍会明显影响总体分数。

6.4 关键实现代码

下面给出 BERT 微调中最关键的三步：tokenizer 编码、前向计算交叉熵，以及依据验证集 Macro-F1 保存最佳状态。完整代码还包含梯度裁剪、学习率调度和 fp16 自动混合精度。

Listing 3: Task 3 的 BERT 微调与验证集选模

```
batch = tokenizer(texts, truncation=True, padding=True,
                  max_length=64, return_tensors="pt")
outputs = model(**batch, labels=labels)
loss = outputs.loss
scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
scaler.step(optimizer)
scaler.update()

with torch.no_grad():
    logits = model(**valid_batch).logits
    valid_pred = logits.argmax(dim=-1).cpu().numpy()
valid_f1 = f1_score(valid_labels, valid_pred, average="macro")
if valid_f1 > best_f1:
    best_f1 = valid_f1
    torch.save(model.state_dict(), best_path)
```

7 综合比较与讨论

表 9: 七组实验的测试集综合比较

任务	方法	Accuracy	Macro-F1
Task 1	Binary BoW	0.989583	0.974058
Task 1	Word Frequency	0.992188	0.980103
Task 1	TF-IDF	0.986111	0.969559
Task 2	GloVe 6B 100d	0.985243	0.964617
Task 2	AG News Word2Vec 100d	0.974826	0.938885
Task 2	NYT Word2Vec 100d	0.982639	0.956510
Task 3	BERT-base-uncased	0.982639	0.960523

从本次固定划分的结果看，Word Frequency 取得最高的 Accuracy 和 Macro-F1，说明该数据集的主题边界主要由高区分度词汇和词频信号决定。词袋模型虽然缺乏语义和词序信息，但在新闻主题分类上具有稀疏、高效和强基线的优势。BERT 具有更强的上下文建模能力，在少数类上保持了较好的平衡，但在本实验的短输入长度和有限训练轮数下没有超过 Word Frequency。BERT 的优势更可能在需要处理否定、实体关系、长距离依赖或多义词的任务中体现。

词向量实验展示了“覆盖率”和“分类性能”并不等价：NYT Word2Vec 的覆盖率最高，却低于 GloVe 的 Macro-F1；AG News Word2Vec 的训练语料规模较大，但跨数据集领域差异导致性能下降。后续可以尝试使用加权平均、最大池化或文档级编码器替代简单平均，以减少重要词被平均操作稀释的问题。

所有结果都基于一次固定的分层划分。虽然固定随机种子保证了复现性，但它不能完全反映不同划分下的方差。若需要进一步提高结论可靠性，可以在保持测试集隔离的前提下进行多次训练集划分、报告均值和标准差，并针对 business 与 politics 的混淆样本进行人工误差分析。

8 心得体会

本实验将文本分类从“调用一个模型得到分数”拆解为数据、表示、分类器和评价四个环节。首先，固定数据划分和随机种子非常重要；如果不同方法使用不同测试样本，比较结果就失去了意义。其次，文本表示的选择应结合任务特点。词袋表示简单、稀疏且容易解释，在主题词明显的新闻分类任务中可以取得很强的效果；词向量把词语映射到连续空间，能够表达一定语义相似性，但简单平均会丢失词序和局部结构。BERT 微调的流程

更加复杂，需要关注 tokenizer 的最大长度、显存、学习率、验证集模型选择和训练过程记录。

实验还让我认识到 Accuracy 不能单独使用。sports 是绝对多数类，仅看 Accuracy 可能掩盖 business 和 politics 的错误；Macro-F1 能够把少数类的表现纳入同等权重。保存逐条预测并进行独立复核也很有价值，它可以发现标签顺序、样本顺序、四舍五入和混淆矩阵计算中的问题，使最终表格具备可追溯性。今后进行类似实验时，我会优先建立统一的数据管线和验证脚本，再开展模型调参，从而减少重复实验和人为记录错误。

9 实验结论

本实验在相同 NYT 三分类数据划分上完成了三项任务。Task 1 中 Word Frequency 表示获得最佳结果，Accuracy 为 0.992188，Macro-F1 为 0.980103。Task 2 中 GloVe 6B 100d 的 Macro-F1 最高，为 0.964617；领域内 NYT Word2Vec 具有最高的测试集词覆盖率 99.1992%。Task 3 中 BERT 第 3 轮验证集 Macro-F1 最优，最终测试集 Accuracy 为 0.982639，Macro-F1 为 0.960523。结果说明，模型复杂度并不必然带来更高的分数，数据分布、表示方式和评价指标共同决定了分类效果；同时，预训练模型为处理更复杂的上下文语义提供了可扩展基础。

Appendix A 代码与结果文件

实验代码和结果文件均位于仓库根目录下，主要文件如下：

表 10: 实验复现所需的主要文件

文件或目录	内容
`scripts/`	数据处理、训练、评估和结果复核脚本
`results/task1\_bow.json`	Task 1 三种词袋方法的汇总结果
`results/task2\_embeddings.json`	Task 2 三种词向量方法的汇总结果
`results/task3\_bert.json`	BERT 最终测试结果与配置
`results/task3\_training\_history.json`	BERT 每轮训练与验证指标
`results/predictions/`	七组模型的逐条验证集和测试集预测
`results/verification.json`	独立复核记录

参考文献

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

[2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[3] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[4] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[5] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1532–1543, 2014.

[6] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.